ASSESSMENT TOOL 3

PSEUDO CODE

SLOT: D **RegisterNo:192424189**

Name: Dhanusha D **Date:24-07-2026**

COURSE: PROGRAMMIN FOR JAVA **Code: DSA0604**

Q1 **Student Management System [BL3 – Apply]** 10 marks

Write pseudocode to design a Student class with attributes (name, rollNo, marks) and methods to:

- Accept student details
- Calculate average marks
- Display result (Pass/Fail)

Apply encapsulation and data hiding by restricting direct access to attributes.

OOP Concepts: Encapsulation | Data Hiding | Classes & Objects

PSEUDO CODE:

Your Answer

```
START
CREATE Class Student
PRIVATE:
   name
   rollNo
   marks1
   marks2
   marks3
METHOD acceptDetails()
   INPUT name
   INPUT rollNo
   INPUT marks1
   INPUT marks2
   INPUT marks3
END METHOD
METHOD calculateAverage()
   average = (marks1 + marks2 + marks3) / 3
   RETURN average
END METHOD
METHOD displayResult()
   average = calculateAverage()
   DISPLAY "Name = ", name
   DISPLAY "Roll No = ", rollNo
   DISPLAY "Average = ", average
```

```
    IF average >= 50 THEN
        DISPLAY "Result = Pass"
    ELSE
        DISPLAY "Result = Fail"
    END IF
END METHOD
END Class
MAIN
CREATE Student object s
CALL s.acceptDetails()
CALL s.displayResult()
STOP
```

Q2 Library Book Management [BL3 – Apply]

Design a Book class with private attributes (title, author, ISBN, isAvailable). Implement methods to:

- Issue a book (change availability status)
- Return a book
- Display book details using getter methods

Demonstrate object creation for at least 3 books and simulate issue/return operations.

OOP Concepts: Encapsulation | Getters & Setters | Objects

PSEUDOCODE:

Your Answer

```
START
CREATE Class Book
PRIVATE:
    title
    author
    ISBN
    isAvailable
METHOD setDetails(t, a, i)
    title = t
    author = a
    ISBN = i
    isAvailable = TRUE
END METHOD
METHOD issueBook()
    IF isAvailable = TRUE THEN
        isAvailable = FALSE
        DISPLAY "Book Issued"
    ELSE
        DISPLAY "Book Not Available"
    END IF
END METHOD
METHOD returnBook()
    isAvailable = TRUE
    DISPLAY "Book Returned"
```

```
END METHOD
METHOD getTitle()
    RETURN title
END METHOD
METHOD getAuthor()
    RETURN author
END METHOD
METHOD getISBN()
    RETURN ISBN
END METHOD
METHOD getStatus()
    RETURN isAvailable
END METHOD
METHOD displayDetails()
    DISPLAY "Title = ", getTitle()
    DISPLAY "Author = ", getAuthor()
    DISPLAY "ISBN = ", getISBN()
    DISPLAY "Available = ", getStatus()
END METHOD
END Class
MAIN
CREATE Book b1
CREATE Book b2
CREATE Book b3
CALL b1.setDetails("Java", "James", "101")
CALL b2.setDetails("Python", "Guido", "102")

CALL b3.setDetails("C++", "Bjarne", "103")
CALL b1.issueBook()
CALL b2.issueBook()
CALL b1.returnBook()
CALL b1.displayDetails()
CALL b2.displayDetails()
CALL b3.displayDetails()
STOP
```

Q3 Employee Salary Calculation [BL3 – Apply]

Write pseudocode for an Employee class with encapsulated attributes (empId, name, basicPay, designation). Implement methods to: • Calculate gross salary (basic + HRA + DA)

• Calculate net salary after deductions (PF, tax)

• Display pay slip

Apply data hiding so salary components are not directly accessible.

OOP Concepts: Encapsulation | Data Hiding | Methods

PSEUDO CODE:

Your Answer

```
START
CREATE Class Employee
PRIVATE:
   empId
   name
   basicPay
   designation
   grossSalary
   netSalary
METHOD setDetails(id, n, pay, des)
   empId = id
   name = n
   basicPay = pay
   designation = des
END METHOD
METHOD calculateGrossSalary()
   HRA = basicPay * 0.20
   DA = basicPay * 0.10
   grossSalary = basicPay + HRA + DA
END METHOD
METHOD calculateNetSalary()
   PF = basicPay * 0.12
   Tax = basicPay * 0.05
   netSalary = grossSalary - PF - Tax
END METHOD
METHOD displayPaySlip()
   DISPLAY "Employee ID = ", empId
   DISPLAY "Name = ", name
   DISPLAY "Designation = ", designation
   DISPLAY "Basic Pay = ", basicPay
   DISPLAY "Gross Salary = ", grossSalary
   DISPLAY "Net Salary = ", netSalary
END METHOD
END Class
MAIN
CREATE Employee e
CALL e.setDetails(101, "Rahul", 30000, "Manager")
CALL e.calculateGrossSalary()
CALL e.calculateNetSalary()
CALL e.displayPaySlip()
STOP
```

Q4 **Shape Area Calculator using Polymorphism [BL3 – Apply]**

Create a Shape base class with an overridable calculateArea() method. Apply this to:

- Circle — area using radius
- Rectangle — area using length and breadth
- Triangle — area using base and height

Use method overriding to demonstrate runtime polymorphism. Call calculateArea() for each object.

OOP Concepts: Inheritance | Polymorphism | Method Overriding

PSEUDO CODE:

Your Answer

```
START
CREATE Class Shape
METHOD calculateArea()
    DISPLAY "Area Calculation"
END METHOD
END Class
CREATE Class Circle EXTENDS Shape
radius
METHOD calculateArea()
    area = 3.14 * radius * radius
    DISPLAY "Circle Area = ", area
END METHOD
END Class
CREATE Class Rectangle EXTENDS Shape
length
breadth
METHOD calculateArea()
    area = length * breadth
    DISPLAY "Rectangle Area = ", area
END METHOD
END Class
CREATE Class Triangle EXTENDS Shape
base
```

```
base
height
METHOD calculateArea()
    area = 0.5 * base * height
    DISPLAY "Triangle Area = ", area
END METHOD
END Class
MAIN
CREATE Circle c
SET c.radius = 5
CREATE Rectangle r
SET r.length = 4
SET r.breadth = 6
CREATE Triangle t
SET t.base = 8
SET t.height = 5
CALL c.calculateArea()
CALL r.calculateArea()
CALL t.calculateArea()
STOP
```

Q5 5. Vehicle Registration System [BL3 – Apply]

Design a Vehicle class with attributes (regNo, owner, fuelType). Extend it into:

- TwoWheeler — with engine capacity
- FourWheeler — with seating capacity and AC status

Apply inheritance to reuse common attributes and override a displayDetails() method in each subclass.

OOP Concepts: Inheritance | Method Overriding | Subclass

PSEUDO CODE:

Your Answer

```
START
CREATE Class Vehicle
regNo
owner
fuelType
METHOD setDetails(r, o, f)
    regNo = r
    owner = o
    fuelType = f
END METHOD
METHOD displayDetails()
    DISPLAY regNo
    DISPLAY owner
    DISPLAY fuelType
END METHOD
END Class
CREATE Class TwoWheeler EXTENDS Vehicle
engineCapacity
METHOD displayDetails()
    DISPLAY regNo
    DISPLAY owner
    DISPLAY fuelType
    DISPLAY engineCapacity
END METHOD
END Class
```

```
CREATE Class FourWheeler EXTENDS Vehicle
seatingCapacity
ACstatus
METHOD displayDetails()
    DISPLAY regNo
    DISPLAY owner
    DISPLAY fuelType
    DISPLAY seatingCapacity
    DISPLAY ACstatus
END METHOD
END Class
MAIN
CREATE TwoWheeler t
CALL t.setDetails("TN01AB1234", "Ravi", "Petrol")
SET t.engineCapacity = 150
CREATE FourWheeler f
CALL f.setDetails("TN02CD5678", "Kumar", "Diesel")
SET f.seatingCapacity = 5
SET f.ACstatus = "Yes"
CALL t.displayDetails()
CALL f.displayDetails()
STOP
```

Q6 6. Bank Account Hierarchy [BL4 – Analyze]

Develop pseudocode for a BankAccount superclass and two subclasses — SavingsAccount (with interest calculation) and CurrentAccount (with overdraft facility). Analyze and implement:

- Inheritance of common attributes (accNo, balance, holder)
- Method overriding for withdrawal behavior
- How overdraft protection differs from savings limit

Compare the behavior of withdraw() across both subclasses and justify the design choices.

OOP Concepts: Inheritance | Method Overriding | Polymorphism

PSEUDO CODE:

Your Answer

```
START
CREATE Class BankAccount
accNo
holder
balance
METHOD setDetails(a, h, b)
   accNo = a
   holder = h
   balance = b
END METHOD
METHOD withdraw(amount)
   DISPLAY "Withdraw Method"
END METHOD
END Class
CREATE Class SavingsAccount EXTENDS BankAccount
METHOD withdraw(amount)
   IF amount <= balance THEN
      balance = balance - amount
      DISPLAY "Amount Withdrawn"
   ELSE
      DISPLAY "Insufficient Balance"
   END IF
END METHOD
METHOD calculateInterest()
```

```
    interest = balance * 5 / 100
    DISPLAY "Interest = ", interest
END METHOD
END Class
CREATE Class CurrentAccount EXTENDS BankAccount
overdraft = 5000
METHOD withdraw(amount)
    IF amount <= balance + overdraft THEN
        balance = balance - amount
        DISPLAY "Amount Withdrawn using Overdraft"
    ELSE
        DISPLAY "Overdraft Limit Exceeded"
    END IF
END METHOD
END Class
MAIN
CREATE SavingsAccount s
CALL s.setDetails(101, "Ravi", 10000)
CREATE CurrentAccount c
CALL c.setDetails(102, "Kumar", 5000)
CALL s.withdraw(3000)
CALL s.calculateInterest()
CALL c.withdraw(8000)
STOP
```

Q7 7. Hospital Patient Record System [BL4 – Analyze] 10 marks

Design a Patient base class and analyze how it extends into:

- InPatient — with ward number, admission date, and daily charges
- OutPatient — with appointment date and consultation fee

Analyze encapsulation requirements for sensitive data (diagnosis, prescription). Override a generateBill() method for each type and compare the billing logic differences. OOP Concepts: Encapsulation | Inheritance | Polymorphism

PSEUDO CODE:

```
START
CREATE Class Patient
PRIVATE:
   patientId
   name
   diagnosis
   prescription
METHOD setDetails(id, n, d, p)
   patientId = id
   name = n
   diagnosis = d
   prescription = p
END METHOD
METHOD generateBill()
   DISPLAY "Bill"
END METHOD
END Class
CREATE Class InPatient EXTENDS Patient
wardNo
admissionDate
dailyCharges
days
METHOD generateBill()
   bill = dailyCharges * days
   DISPLAY "InPatient Bill = ", bill
END METHOD
END Class
CREATE Class OutPatient EXTENDS Patient
appointmentDate
consultationFee
METHOD generateBill()
   bill = consultationFee
   DISPLAY "OutPatient Bill = ", bill
END METHOD
END Class
MAIN
CREATE InPatient ip
CALL ip.setDetails(101, "Ravi", "Fever", "Medicine")
SET ip.wardNo = 12
SET ip.admissionDate = "10-07-2026"
SET ip.dailyCharges = 1000
SET ip.days = 5
CREATE OutPatient op
CALL op.setDetails(102, "Priya", "Cold", "Tablet")
SET op.appointmentDate = "12-07-2026"
SET op.consultationFee = 500
CALL ip.generateBill()
CALL op.generateBill()
STOP
```

Q8 **8. E-Commerce Product Catalog [BL4 – Analyze]** 10 marks

Analyze and design a Product class hierarchy for an e-commerce system:

• Electronics — with warranty, brand, voltage

• Clothing — with size, fabric, gender

• Grocery — with expiryDate and weight

Override an applyDiscount() method for each category with different discount rules. Analyze why polymorphism makes the cart checkout process flexible and extensible. OOP Concepts: Polymorphism | Inheritance | Method Overriding

PSEUDO CODE:

Your Answer

```
warranty
brand
voltage
METHOD applyDiscount()
    discount = price * 10 / 100
    finalPrice = price - discount
    DISPLAY "Electronics Price = ", finalPrice
END METHOD
END Class
CREATE Class Clothing EXTENDS Product
size
fabric
gender
METHOD applyDiscount()
    discount = price * 20 / 100
    finalPrice = price - discount
    DISPLAY "Clothing Price = ", finalPrice
END METHOD
END Class
CREATE Class Grocery EXTENDS Product
expiryDate
weight
METHOD applyDiscount()
    discount = price * 5 / 100
    finalPrice = price - discount
```

```
    DISPLAY "Grocery Price = ", finalPrice
END METHOD
END Class
MAIN
CREATE Electronics e
CALL e.setDetails(101, "Laptop", 50000)
CREATE Clothing c
CALL c.setDetails(102, "Shirt", 1000)
CREATE Grocery g
CALL g.setDetails(103, "Rice", 2000)
CALL e.applyDiscount()
CALL c.applyDiscount()
CALL g.applyDiscount()
STOP
```

Q9 9. University Staff Hierarchy [BL4 – Analyze] 10 marks

Analyze the design of a Staff base class extended by TeachingStaff and NonTeachingStaff. Further extend TeachingStaff into Professor and Lecturer (multilevel inheritance). For each level:

- Identify which attributes should be private vs protected
- Override calculateAllowance() at each level
- Analyze how protected access enables inheritance without full exposure Justify the use of multilevel inheritance and explain how access modifiers enforce data hiding.

PSEUDO CODE:

Your Answer

```
START
CREATE Class Staff
PRIVATE:
   staffId
   name
PROTECTED:
   basicSalary
METHOD setDetails(id, n, salary)
   staffId = id
   name = n
   basicSalary = salary
END METHOD
METHOD calculateAllowance()
   DISPLAY "Allowance"
END METHOD
END Class
CREATE Class TeachingStaff EXTENDS Staff
subject
METHOD calculateAllowance()
   allowance = basicSalary * 20 / 100
   DISPLAY "Teaching Allowance = ", allowance
END METHOD
END Class
CREATE Class Professor EXTENDS TeachingStaff
METHOD calculateAllowance()
   allowance = basicSalary * 15 / 100
   DISPLAY "Lecturer Allowance = ", allowance
END METHOD
END Class
CREATE Class NonTeachingStaff EXTENDS Staff
department
METHOD calculateAllowance()
   allowance = basicSalary * 10 / 100
   DISPLAY "Non-Teaching Allowance = ", allowance
END METHOD
END Class
MAIN
CREATE Professor p
CALL p.setDetails(101, "Ravi", 50000)
CALL p.calculateAllowance()
CREATE Lecturer l
CALL l.setDetails(102, "Priya", 30000)
CALL l.calculateAllowance()
CREATE NonTeachingStaff n
CALL n.setDetails(103, "Kumar", 25000)
CALL n.calculateAllowance()
STOP
```

Q10 **10. Smart Home Device Controller [BL4 – Analyze]** 10 marks

Analyze and design a SmartDevice superclass with subclasses: SmartLight, SmartThermostat, and SmartLock. Override an executeCommand(cmd) method in each, where the same command (e.g., 'on', 'off', 'status') produces device-specific behavior. Analyze:

• How polymorphism allows a single controller loop to manage all devices

• Security implications of data hiding in SmartLock (PIN must never be exposed) Design the class hierarchy and explain the OOP principles applied at each level.

OOP Concepts: Polymorphism | Encapsulation | Inheritance

PSEUDO CODE:

Your Answer

```
START
CREATE Class SmartDevice
deviceName
METHOD executeCommand(cmd)
   DISPLAY "Command Executed"
END METHOD
END Class
CREATE Class SmartLight EXTENDS SmartDevice
METHOD executeCommand(cmd)
   IF cmd = "on" THEN
      DISPLAY "Light ON"
   ELSE IF cmd = "off" THEN
      DISPLAY "Light OFF"
   ELSE
      DISPLAY "Light Status"
   END IF
END METHOD
END Class
CREATE Class SmartThermostat EXTENDS SmartDevice
METHOD executeCommand(cmd)
   IF cmd = "on" THEN
      DISPLAY "Thermostat ON"
   ELSE IF cmd = "off" THEN
      DISPLAY "Thermostat OFF"
```

```
    ELSE
        DISPLAY "Temperature Status"
    END IF
END METHOD
END Class
CREATE Class SmartLock EXTENDS SmartDevice
PRIVATE:
    PIN
METHOD executeCommand(cmd)
    IF cmd = "on" THEN
        DISPLAY "Door Locked"
    ELSE IF cmd = "off" THEN
        DISPLAY "Door Unlocked"
    ELSE
        DISPLAY "Lock Status"
    END IF
END METHOD
END Class
MAIN
CREATE SmartLight L
CREATE SmartThermostat T
CREATE SmartLock S
CALL L.executeCommand("on")
CALL T.executeCommand("status")
CALL S.executeCommand("off")
STOP
```